



Advanced Python programming using AI tools

Lecture 9: GitHub Copilot & OOP

Lecturer: Dr. Havana Rika

Today

1. Basic Object-Oriented Programming
2. GitHub Copilot



Procedural Programming

- Procedural programming: writing programs made of functions that perform specific tasks
 - Procedures typically operate on data items that are separate from the procedures
 - Data items commonly passed from one procedure to another
 - Focus: to create procedures that operate on the program's data

Object-Oriented Programming

- Object-oriented programming: focused on creating objects
- Object: entity that contains data and procedures
 - Data is known as data attributes and procedures are known as methods
 - Methods perform operations on the data attributes
- Encapsulation: combining data and code into a single object

Object-Oriented Programming

- Data hiding: object's data attributes are hidden from code outside the object
 - Access restricted to the object's methods
 - Protects from accidental corruption
 - Outside code does not need to know internal structure of the object
- Object reusability: the same object can be used in different programs
 - Example: 3D image object can be used for architecture and game programming

An Everyday Example of an Object

- Data attributes: define the state of an object
 - Example: clock object would have `second`, `minute`, and `hour` data attributes
- Public methods: allow external code to manipulate the object
 - Example: `set_time`, `set_alarm_time`
- Private methods: used for object's inner workings

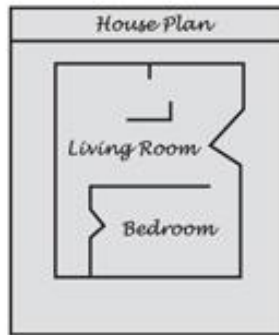
Classes

- Class: code that specifies the data attributes and methods of a particular type of object
 - Similar to a blueprint of a house or a cookie cutter
- Instance: an object created from a class
 - Similar to a specific house built according to the blueprint or a specific cookie
 - There can be many instances of one class



Classes

Blueprint that describes a house



Instances of the house described by the blueprint



Figure 10-3 A blueprint and houses built from the blueprint

Class Definitions

- Class definition: set of statements that define a class's methods and data attributes
 - Format: begin with `class Class_name:`
 - Class names often start with uppercase letter
 - Method definition like any other python function definition
 - self parameter: required in every method in the class – references the specific object that the method is working on

Class Definitions

- Initializer method: automatically executed when an instance of the class is created
 - Initializes object's data attributes and assigns `self` parameter to the object that was just created
 - Format: `def __init__(self):`
 - Usually the first method in a class definition

Example

```
1 import random
2
3 # The Coin class simulates a coin that can
4 # be flipped.
5
6 class Coin:
7
8     # The __init__ method initializes the
9     # sideup data attribute with 'Heads'.
10
11     def __init__(self):
12         self.sideup = 'Heads'
13
14     # The toss method generates a random number
15     # in the range of 0 through 1. If the number
16     # is 0, then sideup is set to 'Heads'.
17     # Otherwise, sideup is set to 'Tails'.
18
19     def toss(self):
20         if random.randint(0, 1) == 0:
21             self.sideup = 'Heads'
22         else:
23             self.sideup = 'Tails'
24
25     # The get_sideup method returns the value
26     # referenced by sideup.
27
28     def get_sideup(self):
29         return self.sideup
30
```

Program 10-1 (Coin class, not a complete program)

Example (Complete)

Program 10-2 (coin_demo1.py)

```
1 import random
2
3 # The Coin class simulates a coin that can
4 # be flipped.
5
6 class Coin:
7
8     # The __init__ method initializes the
9     # sideup data attribute with 'Heads'.
10
11     def __init__(self):
12         self.sideup = 'Heads'
13
14     # The toss method generates a random number
15     # in the range of 0 through 1. If the number
16     # is 0, then sideup is set to 'Heads'.
17     # Otherwise, sideup is set to 'Tails'.
18
19     def toss(self):
20         if random.randint(0, 1) == 0:
21             self.sideup = 'Heads'
22         else:
23             self.sideup = 'Tails'
24
25     # The get_sideup method returns the value
26     # referenced by sideup.
27
28     def get_sideup(self):
29         return self.sideup
30
31 # The main function.
32 def main():
33     # Create an object from the Coin class.
34     my_coin = Coin()
35
36     # Display the side of the coin that is facing up.
37     print('This side is up:', my_coin.get_sideup())
38
39     # Toss the coin.
40     print('I am tossing the coin ...')
41     my_coin.toss()
42
43     # Display the side of the coin that is facing up.
44     print('This side is up:', my_coin.get_sideup())
45
46 # Call the main function.
47 if __name__ == '__main__':
48     main()
```

Example (Complete)

Program 10-2 (coin_demo1.py)

```
1 import random
2
3 # The Coin class simulates a coin that can
4 # be flipped.
5
6 class Coin:
7
8     # The __init__ method initializes the
9     # sideup data attribute with 'Heads'.
10
11     def __init__(self):
12         self.sideup = 'Heads'
13
14     # The toss method generates a random number
15     # in the range of 0 through 1. If the number
16     # is 0, then sideup is set to 'Heads'.
17     # Otherwise, sideup is set to 'Tails'.
18
19     def toss(self):
20         if random.randint(0, 1) == 0:
21             self.sideup = 'Heads'
22         else:
23             self.sideup = 'Tails'
24
25     # The get_sideup method returns the value
26     # referenced by sideup.
27
28     def get_sideup(self):
29         return self.sideup
30
31 # The main function.
32 def main():
33     # Create an object from the Coin class.
34     my_coin = Coin()
35
36     # Display the side of the coin that is facing up.
37     print('This side is up:', my_coin.get_sideup())
38
39     # Toss the coin.
40     print('I am tossing the coin ...')
41     my_coin.toss()
42
43     # Display the side of the coin that is facing up.
44     print('This side is up:', my_coin.get_sideup())
45
46 # Call the main function.
47 if __name__ == '__main__':
48     main()
```

Program Output

This side is up: Heads
I am tossing the coin ...
This side is up: Tails

Program Output

This side is up: Heads
I am tossing the coin ...
This side is up: Heads

Program Output

This side is up: Heads
I am tossing the coin ...
This side is up: Tails

Class Definitions (3 of 4)

1 An object is created in memory from the `Coin` class.

2 The `Coin` class's `__init__` method is called, and the `self` parameter is set to the newly created object

After these steps take place, a `Coin` object will exist with its `sideup` attribute set to `'Heads'`.

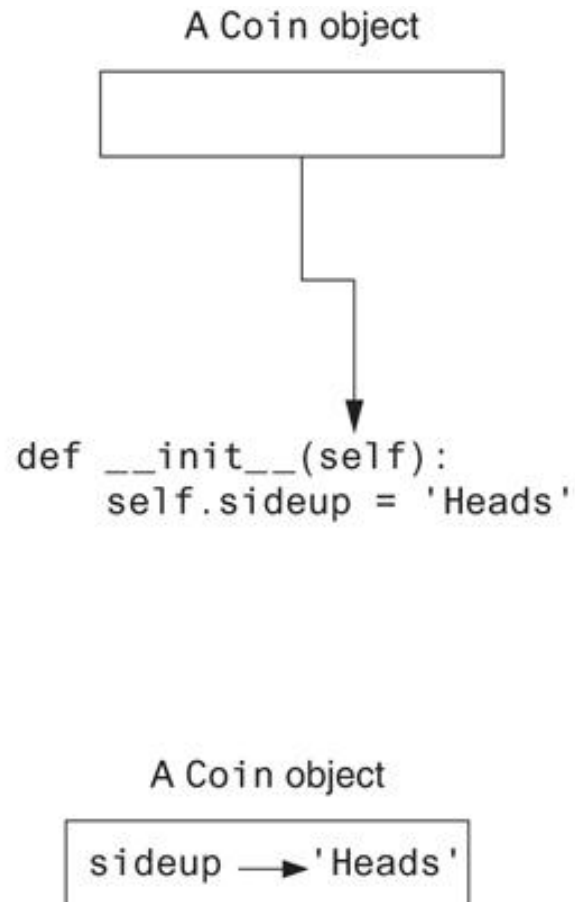


Figure 10-5 Actions caused by the `Coin()` expression

Class Definitions

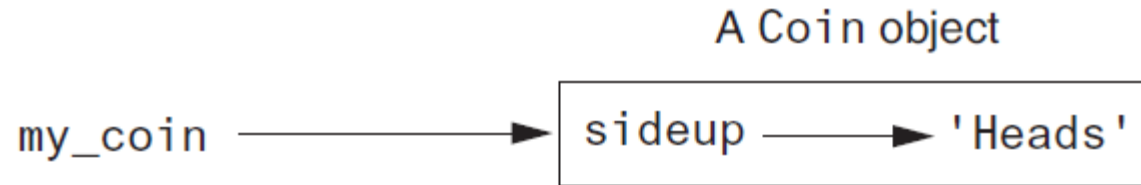


Figure 10-6 The `my_coin` variable references a `Coin` object

Class Definitions (4 of 4)

- To create a new instance of a class call the initializer method
 - Format: *My_instance = Class_Name()*
- To call any of the class methods using the created instance, use dot notation
 - Format: *My_instance.method()*
 - Because the `self` parameter references the specific instance of the object, the method will affect this instance
 - Reference to `self` is passed automatically

Hiding Attributes and Storing Classes in Modules

- An object's data attributes should be private
 - To make sure of this, place two underscores (__) in front of attribute name
 - Example: `__current_minute`
- Classes can be stored in modules
 - Filename for module must end in `.py`
 - Module can be imported to programs that use the class

Example

```
1 import random
2
3 # The Coin class simulates a coin that can
4 # be flipped.
5
6 class Coin:
7
8     # The __init__ method initializes the
9     # sideup data attribute with 'Heads'.
10
11     def __init__(self):
12         self.sideup = 'Heads'
13
14     # The toss method generates a random number
15     # in the range of 0 through 1. If the number
16     # is 0, then sideup is set to 'Heads'.
17     # Otherwise, sideup is set to 'Tails'.
18
19     def toss(self):
20         if random.randint(0, 1) == 0:
21             self.sideup = 'Heads'
22         else:
23             self.sideup = 'Tails'
24
25     # The get_sideup method returns the value
26     # referenced by sideup.
27
28     def get_sideup(self):
29         return self.sideup
30
31 # The main function.
32 def main():
33     # Create an object from the Coin class.
34     my_coin = Coin()
35
36     # Display the side of the coin that is facing up.
37     print('This side is up:', my_coin.get_sideup())
38
39     # Toss the coin.
40     print('I am tossing the coin ...')
41     my_coin.toss()
42
43     # But now I'm going to cheat! I'm going to
44     # directly change the value of the object's
45     # sideup attribute to 'Heads'.
46     my_coin.sideup = 'Heads'
47
48     # Display the side of the coin that is facing up.
49     print('This side is up:', my_coin.get_sideup())
50
51 # Call the main function.
52 if __name__ == '__main__':
53     main()
```

Program 10-3 (coin_demo2.py)

Program Output

This side is up: Heads
I am tossing the coin ...
This side is up: Heads

Program Output

This side is up: Heads
I am tossing the coin ...
This side is up: Heads

Program Output

This side is up: Heads
I am tossing the coin ...
This side is up: Heads

Example

Program 10-5 (coin.py)

```
1 import random
2
3 # The Coin class simulates a coin that can
4 # be flipped.
5
6 class Coin:
7
8     # The __init__ method initializes the
9     # sideup data attribute with 'Heads'.
10
11     def __init__(self):
12         self.__sideup = 'Heads'
13
14     # The toss method generates a random number
15     # in the range of 0 through 1. If the number
16     # is 0, then sideup is set to 'Heads'.
17     # Otherwise, sideup is set to 'Tails'.
18
19     def toss(self):
20         if random.randint(0, 1) == 0:
21             self.__sideup = 'Heads'
22         else:
23             self.__sideup = 'Tails'
24
25     # The get_sideup method returns the value
26     # referenced by sideup.
27
28     def get_sideup(self):
29         return self.__sideup
30
```

Program 10-6 (coin_demo4.py)

```
1 # This program imports the coin module and
2 # creates an instance of the Coin class.
3
4 import coin
5
6 def main():
7     # Create an object from the Coin class.
8     my_coin = coin.Coin()
9
10    # Display the side of the coin that is facing up.
11    print('This side is up:', my_coin.get_sideup())
12
13    # Toss the coin.
14    print('I am going to toss the coin ten times:')
15    for count in range(10):
16        my_coin.toss()
17        print(my_coin.get_sideup())
18
19 # Call the main function.
20 if __name__ == '__main__':
21     main()
```

Program Output

```
This side is up: Heads
I am going to toss the coin ten times:
Tails
Tails
Heads
Tails
Heads
...
```

Private Method

```
1 import random
2
3 class Coin: 1 usage
4
5     def __init__(self):
6         self.__sideup = 'Heads'
7
8     def __toss(self):
9         if random.randint(a: 0, b: 1) == 0:
10             self.__sideup = 'Heads'
11         else:
12             self.__sideup = 'Tails'
13
14     def get_sideup(self): 2 usages
15         return self.__sideup
```

```
1 import coin
2
3 def main(): 1 usage
4     my_coin = coin.Coin()
5
6     print('This side is up:', my_coin.get_sideup())
7
8     print('I am going to toss the coin ten times:')
9     for count in range(10):
10         my_coin.__toss()
11         print(my_coin.get_sideup())
12
13 if __name__ == '__main__':
14     main()
```

```
C:\Users\Havana\anaconda3\envs\MLDSenv\python.exe C:\Users\Havana\PycharmProjects\PythonCourse\program.py
Traceback (most recent call last):
  File "C:\Users\Havana\PycharmProjects\PythonCourse\program.py", line 21, in <module>
    main()
  File "C:\Users\Havana\PycharmProjects\PythonCourse\program.py", line 16, in main
    my_coin.__toss()
    ^^^^^^^^^^^^^^^^^
AttributeError: 'Coin' object has no attribute '__toss'
This side is up: Heads
I am going to toss the coin ten times:

Process finished with exit code 1
```

Private Attribute

```
1 import random
2
3 class Coin: 1 usage
4
5     def __init__(self):
6         self.__sideup = 'Heads'
7
8     def toss(self): 2 usages
9         if random.randint(a: 0, b: 1) == 0:
10             self.__sideup = 'Heads'
11         else:
12             self.__sideup = 'Tails'
13
14     def get_sideup(self): 2 usages
15         return self.__sideup
```

```
1 import coin
2
3 def main(): 1 usage
4     my_coin = coin.Coin()
5
6     while my_coin.toss()=="Talis":
7         my_coin.toss()
8
9     print('This side is up:', my_coin.get_sideup())
10
11     my_coin.__sideup = 'Heads'
12
13     print('This side is up:', my_coin.get_sideup())
14
15 ► if __name__ == '__main__':
16     main()
```

This side is up: Tails

This side is up: Tails

Process finished with exit code 0

The BankAccount Class – More About Classes

- Class methods can have multiple parameters in addition to `self`
 - For `__init__`, parameters needed to create an instance of the class
 - Example: a `BankAccount` object is created with a balance
 - When called, the initializer method receives a value to be assigned to a `__balance` attribute
 - For other methods, parameters needed to perform required task
 - Example: `deposit` method amount to be deposited

The BankAccount Class

Program 10-7 (bankaccount.py)

```
1 # The BankAccount class simulates a bank account.
2
3 class BankAccount:
4
5     # The __init__ method accepts an argument for
6     # the account's balance. It is assigned to
7     # the __balance attribute.
8
9     def __init__(self, bal):
10         self.__balance = bal
11
12     # The deposit method makes a deposit into the
13     # account.
14
15     def deposit(self, amount):
16         self.__balance += amount
17
18     # The withdraw method withdraws an amount
19     # from the account.
20
21     def withdraw(self, amount):
22         if self.__balance >= amount:
23             self.__balance -= amount
24         else:
25             print('Error: Insufficient funds')
26
27     # The get_balance method returns the
28     # account balance.
29
30     def get_balance(self):
31         return self.__balance
```

Program 10-8 (account_test.py)

```
1 # This program demonstrates the BankAccount class.
2
3 import bankaccount
4
5 def main():
6     # Get the starting balance.
7     start_bal = float(input('Enter your starting balance: '))
8
9     # Create a BankAccount object.
10    savings = bankaccount.BankAccount(start_bal)
11
12    # Deposit the user's paycheck.
13    pay = float(input('How much were you paid this week? '))
14    print('I will deposit that into your account.')
15    savings.deposit(pay)
16
17    # Display the balance.
18    print(f'Your account balance is ${savings.get_balance():.2f}.')
19
20    # Get the amount to withdraw.
21    cash = float(input('How much would you like to withdraw? '))
22    print('I will withdraw that from your account.')
23    savings.withdraw(cash)
24
25    # Display the balance.
26    print(f'Your account balance is ${savings.get_balance():.2f}.')
27
28 # Call the main function.
29 if __name__ == '__main__':
30     main()
```

The BankAccount Class

Program 10-7 (bankaccount.py)

```
1 # The BankAccount class simulates a bank account.
2
3 class BankAccount:
4
5     # The __init__ method accepts an argument for
6     # the account's balance. It is assigned to
7     # the __balance attribute.
8
9     def __init__(self, bal):
10         self.__balance = bal
11
12     # The deposit method makes a deposit into the
13     # account.
14
15     def deposit(self, amount):
16         self.__balance += amount
17
18     # The withdraw method withdraws an amount
19     # from the account.
20
21     def withdraw(self, amount):
22         if self.__balance >= amount:
23             self.__balance -= amount
24         else:
25             print('Error: Insufficient funds')
26
27     # The get_balance method returns the
28     # account balance.
29
30     def get_balance(self):
31         return self.__balance
```

Program 10-8 (account_test.py)

```
1 # This program demonstrates the BankAccount class.
2
3 import bankaccount
4
5 def main():
6     # Get the starting balance.
7     start_bal = float(input('Enter your starting balance: '))
8
9     # Create a BankAccount object.
10    savings = bankaccount.BankAccount(start_bal)
11
12    # Deposit the user's paycheck.
13    pay = float(input('How much were you paid this week? '))
14    print('I will deposit that into your account.')
15    savings.deposit(pay)
16
17    # Display the balance.
18    print(f'Your account balance is ${savings.get_balance():.2f}.')
19
20    # Get the amount to withdraw.
21    cash = float(input('How much would you like to withdraw? '))
22    print('I will withdraw that from your account.')
23    savings.withdraw(cash)
24
25    # Display the balance.
26    print(f'Your account balance is ${savings.get_balance():.2f}.')
27
28 # Call the main function.
29 if __name__ == '__main__':
30     main()
```

Program Output (with input shown in bold)
Enter your starting balance: **1000.00**
How much were you paid this week? **500.00**
I will deposit that into your account.
Your account balance is \$1,500.00
How much would you like to withdraw? **2000.00**.
I will withdraw that from your account.
Error: Insufficient funds
Your account balance is \$1,500.00

BankAccount Class with Default C'tor

Program 10-7 (bankaccount.py)

```
1 # The BankAccount class simulates a bank account.
2
3 class BankAccount:
4
5     # The __init__ method accepts an argument for
6     # the account's balance. It is assigned to
7     # the __balance attribute.
8
9     def __init__(self, bal=0):
10         self.__balance = bal
11
12     # The deposit method makes a deposit into the
13     # account.
14
15     def deposit(self, amount):
16         self.__balance += amount
17
18     # The withdraw method withdraws an amount
19     # from the account.
20
21     def withdraw(self, amount):
22         if self.__balance >= amount:
23             self.__balance -= amount
24         else:
25             print('Error: Insufficient funds')
26
27     # The get_balance method returns the
28     # account balance.
29
30     def get_balance(self):
31         return self.__balance
```

Program 10-8 (account_test.py)

```
1 # This program demonstrates the BankAccount class.
2
3 import bankaccount
4
5 def main():
6
7
8     # Create a BankAccount object.
9     savings = bankaccount.BankAccount()
10
11
12     # Deposit the user's paycheck.
13     pay = float(input('How much were you paid this week? '))
14     print('I will deposit that into your account.')
15     savings.deposit(pay)
16
17     # Display the balance.
18     print(f'Your account balance is ${savings.get_balance():.2f}.')
19
20     # Get the amount to withdraw.
21     cash = float(input('How much would you like to withdraw? '))
22     print('I will withdraw that from your account.')
23     savings.withdraw(cash)
24
25     # Display the balance.
26     print(f'Your account balance is ${savings.get_balance():.2f}.')
27
28 # Call the main function.
29 if __name__ == '__main__':
30     main()
```

The `__str__` method

- Object's state: the values of the object's attribute at a given moment
- `__str__` method: displays the object's state
 - Automatically called when the object is passed as an argument to the `print` function
 - Automatically called when the object is passed as an argument to the `str` function

The BankAccount Class (with str method)

Program 10-9 (bankaccount2.py)

```
1 # The BankAccount class simulates a bank account.
2
3 class BankAccount:
4
5     # The __init__ method accepts an argument for
6     # the account's balance. It is assigned to
7     # the __balance attribute.
8
9     def __init__(self, bal):
10         self.__balance = bal
11
12     # The deposit method makes a deposit into the
13     # account.
14
15     def deposit(self, amount):
16         self.__balance += amount
17
18     # The withdraw method withdraws an amount
19     # from the account.
20
21     def withdraw(self, amount):
22         if self.__balance >= amount:
23             self.__balance -= amount
24         else:
25             print('Error: Insufficient funds')
26
27     # The get_balance method returns the
28     # account balance.
29
30     def get_balance(self):
31         return self.__balance
32
33     # The __str__ method returns a string
34     # indicating the object's state.
35
36     def __str__(self):
37         return f'The balance is ${self.__balance:,.2f}'
```

Program 10-10 (account_test2.py)

```
1 # This program demonstrates the BankAccount class
2 # with the __str__ method added to it.
3
4 import bankaccount2
5
6 def main():
7     # Get the starting balance.
8     start_bal = float(input('Enter your starting balance: '))
9
10    # Create a BankAccount object.
11    savings = bankaccount2.BankAccount(start_bal)
12
13    # Deposit the user's paycheck.
14    pay = float(input('How much were you paid this week? '))
15    print('I will deposit that into your account.')
16    savings.deposit(pay)
17
18    # Display the balance.
19    print(savings)
20
21    # Get the amount to withdraw.
22    cash = float(input('How much would you like to withdraw? '))
23    print('I will withdraw that from your account.')
24    savings.withdraw(cash)
25
26    # Display the balance.
27    print(savings)
28
29 # Call the main function.
30 if __name__ == '__main__':
31     main()
```

Program Output (with input shown in bold)

```
Enter your starting balance: 1000.00
How much were you paid this week? 500.00
I will deposit that into your account.
The account balance is $1,500.00
How much would you like to withdraw? 1200.00
I will withdraw that from your account.
The account balance is $300.00
```

Working With Instances (Example)

Program 10-11 (coin_demo5.py)

```
1 # This program imports the simulation module and
2 # creates three instances of the Coin class.
3
4 import coin
5
6 def main():
7     # Create three objects from the Coin class.
8     coin1 = coin.Coin()
9     coin2 = coin.Coin()
10    coin3 = coin.Coin()
11
12    # Display the side of each coin that is facing up.
13    print('I have three coins with these sides up:')
14    print(coin1.get_sideup())
15    print(coin2.get_sideup())
16    print(coin3.get_sideup())
17    print()
18
19    # Toss the coin.
20    print('I am tossing all three coins ...')
21    print()
22    coin1.toss()
23    coin2.toss()
24    coin3.toss()
25
26    # Display the side of each coin that is facing up.
27    print('Now here are the sides that are up:')
28    print(coin1.get_sideup())
29    print(coin2.get_sideup())
30    print(coin3.get_sideup())
31    print()
32
33 # Call the main function.
34 if __name__ == '__main__':
35     main()
```

Program Output

```
I have three coins with these sides up:
Heads
Heads
Heads
I am tossing all three coins ...
Now here are the sides that are up:
Tails
Tails
Heads
```

Working With Instances

- Instance attribute: belongs to a specific instance of a class
 - Created when a method uses the `self` parameter to create an attribute
- If many instances of a class are created, each would have its own set of attributes

Working With Instances

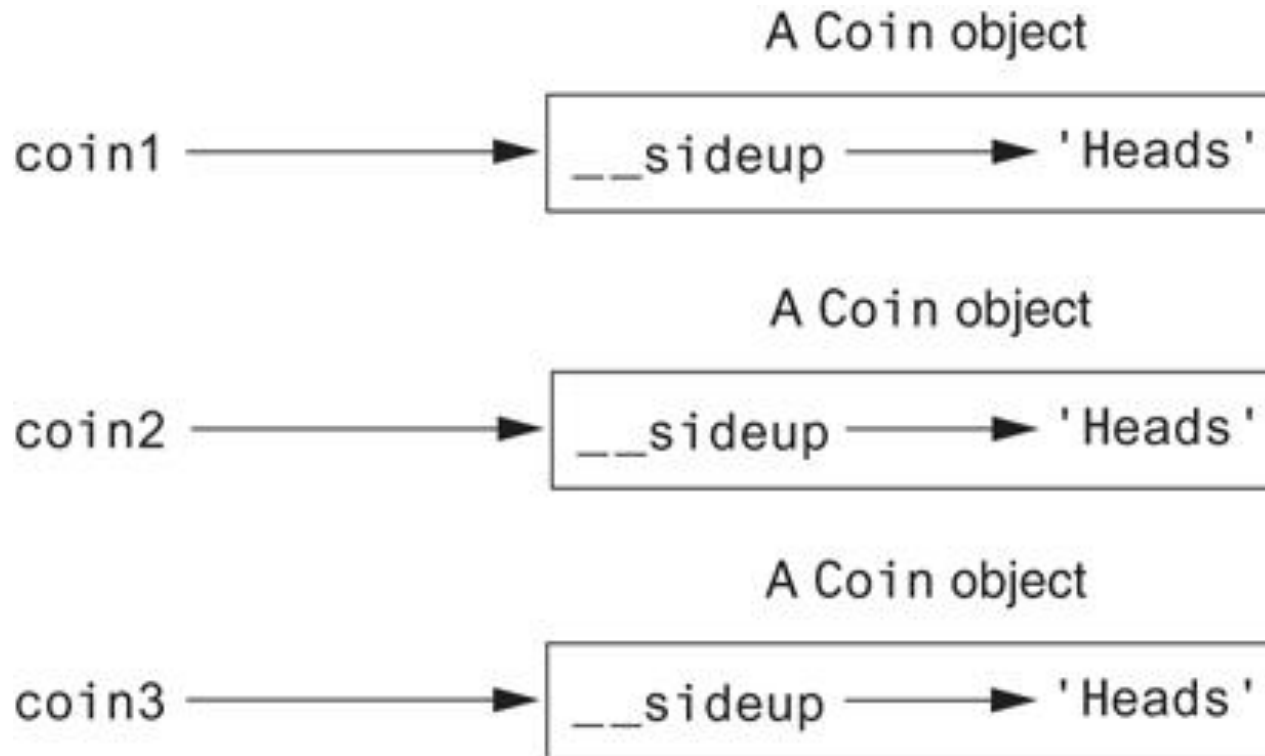


Figure 10-7 The `coin1`, `coin2`, and `coin3` variables reference three `Coin` objects

Working With Instances

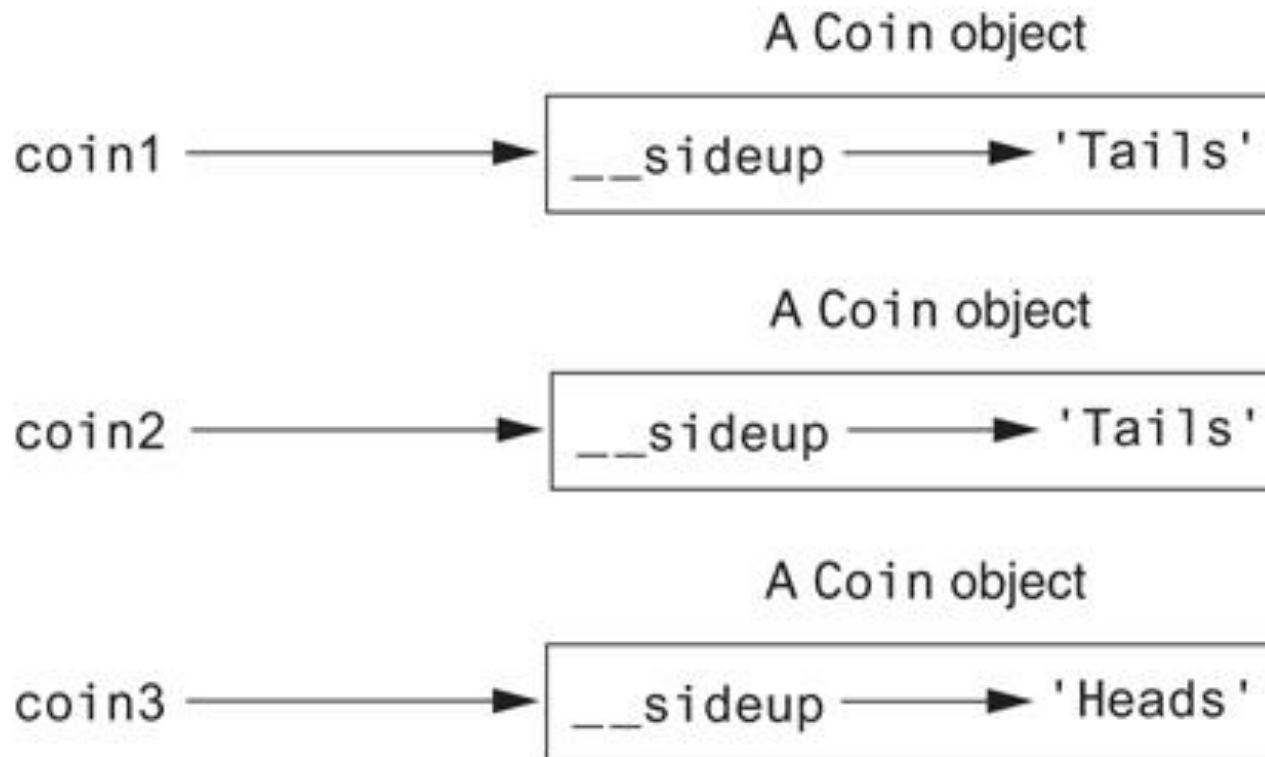


Figure 10-8 The objects after the `toss` method

Set and Get Methods (Example)

Program 10-12 (cellphone.py)

```
1 # The CellPhone class holds data about a cell phone.
2
3 class CellPhone:
4
5     # The __init__ method initializes the attributes.
6
7     def __init__(self, manufact, model, price):
8         self.__manufact = manufact
9         self.__model = model
10        self.__retail_price = price
11
12    # The set_manufact method accepts an argument
13    # for the phone's manufacturer.
14
15    def set_manufact(self, manufact):
16        self.__manufact = manufact
17
18    # The set_model method accepts an argument for
19    # the phone's model number.
20
21    def set_model(self, model):
22        self.__model = model
23
24    # The set_retail_price method accepts an argument
25    # for the phone's retail price.
26
27    def set_retail_price(self, price):
28        self.__retail_price = price
29
30    # The get_manufact method returns the
31    # phone's manufacturer.
32
33    def get_manufact(self):
34        return self.__manufact
35
36    # The get_model method returns the
37    # phone's model number.
38
39    def get_model(self):
40        return self.__model
41
42    # The get_retail_price method returns the
43    # phone's retail price.
44
45    def get_retail_price(self):
46        return self.__retail_price
```


Set and Get Methods (Example)

Program 10-13 (cell_phone_test.py)

```
1 # This program tests the CellPhone class.
2
3 import cellphone
4
5 def main():
6     # Get the phone data.
7     man = input('Enter the manufacturer: ')
8     mod = input('Enter the model number: ')
9     retail = float(input('Enter the retail price: '))
10
11 # Create an instance of the CellPhone class.
12 phone = cellphone.CellPhone(man, mod, retail)
13
14 # Display the data that was entered.
15 print('Here is the data that you entered:')
16 print(f'Manufacturer: {phone.get_manufact()}')
17 print(f'Model Number: {phone.get_model()}')
18 print(f'Retail Price: ${phone.get_retail_price():,.2f}')
19
20 # Call the main function.
21 if __name__ == '__main__':
22     main()
```

Program Output (with input shown in bold)
Enter the manufacturer: **Acme Electronics**
Enter
Enter the model number: **M1000** **Enter**
Enter the retail price: **199.99** **Enter**
Here is the data that you entered:
Manufacturer: Acme Electronics
Model Number: M1000
Retail Price: \$199.99

Set and Get Methods

- Typically, all of a class's data attributes are private and provide methods to access and change them
- Get methods: return a value from a class's attribute without changing it
 - Safe way for code outside the class to retrieve the value of attributes
- Set methods: store or change the value of a data attribute

Passing Objects as Arguments

```
1 # This program passes a Coin object as
2 # an argument to a function.
3 import coin
4
5 # main function
6 def main():
7     # Create a Coin object.
8     my_coin = coin.Coin()
9
10    # This will display 'Heads'.
11    print(my_coin.get_sideup())
12
13    # Pass the object to the flip function.
14    flip(my_coin)
15
16    # This might display 'Heads', or it might
17    # display 'Tails'.
18    print(my_coin.get_sideup())
19
20 # The flip function flips a coin.
21 def flip(coin_obj):
22     coin_obj.toss()
23
24 # Call the main function.
25 if __name__ == '__main__':
26     main()
```

Program Output

Heads

Tails

Program Output

Heads

Heads

Program Output

Heads

Tails

Passing Objects as Arguments

- Methods and functions often need to accept objects as arguments
- When you pass an object as an argument, you are actually passing a reference to the object
 - The receiving method or function has access to the actual object
 - Methods of the object can be called within the receiving function or method, and data attributes may be changed using mutator methods

Inheritance

```
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def printname(self):
        print(self.firstname, self.lastname)
```

- אם המשתנים פרטיים אצל האבא אז לבן אין גישה אליהם.

- קיים מנגנון דריסה בפייתון, מתבצעת דריסה כאשר מגדירים בבן מתודה בעלת אותו השם כמו אצל האבא (גם אם פרמטרי הקלט שונים).

- המחלקות יכולות להיות באותו הקובץ או בקבצים נפרדים.

```
class Student(Person):
    def __init__(self, fname, lname, year):
        super().__init__(fname, lname) #Person.__init__(self, fname, lname)
        self.graduationyear = year

    def welcome(self):
        print("Welcome", self.firstname, self.lastname, "to the class of", self.graduationyear)
```

```
x = Student("Mike", "Olsen", 2019)
x.welcome()
```

Inheritance

```
class Person:
    def __init__(self, fname, lname, id):
        self.__firstname = fname
        self.__lastname = lname
        self.__id = id

    def get_firstname(self):
        return self.__firstname

    def set_firstname(self, fname):
        self.__firstname = fname

    def get_lastname(self):
        return self.__lastname

    def set_lastname(self, lname):
        self.__lastname = lname

    def get_id(self):
        return self.__id

    def __str__(self):
        return f"firstname={self.__firstname}, lastname={self.__lastname}"
```

Inheritance

```
class Student(Person):
    def __init__(self, fname, lname, id, year):
        super().__init__(fname, lname, id)
        self.__graduationyear = year

    def welcome(self):
        print("Welcome", self.get_firstname(), self.get_lastname(), "to the class of",
self.__graduationyear)

    def get_graduationyear(self):
        return self.__graduationyear

    def set_graduationyear(self, year):
        self.__graduationyear = year

    def __str__(self):
        return super().__str__() + f", graduationyear={self.__graduationyear}"
```

```
x = Student("Mike", "Olsen", 123456789, 2019)
x.welcome()
print(x)
```

```
Welcome Mike Olsen to the class of 2019
firstname=Mike, lastname=Olsen, graduationyear=2019
```

Customized Exception

```
1 class InvalidAmount(Exception): 3 usages
2     def __init__(self, message):
3         super().__init__(message)
4
5 class Bank_Account: 1 usage
6     def __init__(self, balance=0):
7         self.__check_amount(balance)
8         self.__balance = balance
9
10    def deposit(self, amount): 1 usage
11        self.__check_amount(amount)
12        self.__balance += amount
13
14    def withdraw(self, amount): 1 usage
15        self.__check_amount(amount)
16        if self.__balance >= amount:
17            self.__balance -= amount
18        else:
19            print('Error: Insufficient funds')
20
21    def get_balance(self):
22        return self.__balance
23
24    def __check_amount(self, amount): 3 usages
25        try:
26            amount = float(amount)
27        except ValueError:
28            raise InvalidAmount('Amount must be a number')
29        if amount < 0:
30            raise InvalidAmount('Amount must be greater than zero')
31
32    def __str__(self):
33        return f'The balance is ${self.__balance:,.2f}'
```


Customized Exception

```
1 class InvalidAmount(Exception): 3 usages
2     def __init__(self, message):
3         super().__init__(message)
4
5 class Bank_Account: 1 usage
6     def __init__(self, balance=0):
7         self.__check_amount(balance)
8         self.__balance = balance
9
10    def deposit(self, amount): 1 usage
11        self.__check_amount(amount)
12        self.__balance += amount
13
14    def withdraw(self, amount): 1 usage
15        self.__check_amount(amount)
16        if self.__balance >= amount:
17            self.__balance -= amount
18        else:
19            print('Error: Insufficient funds')
20
21    def get_balance(self):
22        return self.__balance
23
24    def __check_amount(self, amount): 3 usages
25        try:
26            amount = float(amount)
27        except ValueError:
28            raise InvalidAmount('Amount must be a number')
29        if amount < 0:
30            raise InvalidAmount('Amount must be greater than zero')
31
32    def __str__(self):
33        return f'The balance is ${self.__balance:,.2f}'
```

```
import bank_account

def main(): 1 usage

    try:
        my_account = bank_account.Bank_Account(1000)
        print(my_account)

        print("Depositing $500:")
        my_account.deposit(500)
        print(my_account)

        print("Withdrawing $-200:")
        my_account.withdraw(-200)
        print(my_account)

    except bank_account.InvalidAmount as e:
        print(f"Invalid amount error: {e}")

if __name__ == '__main__':
    main()
```

```
The balance is $1,000.00
Depositing $500:
The balance is $1,500.00
Withdrawing $-200:
Invalid amount error: Amount must be greater than zero

Process finished with exit code 0
```

Customized Exception

```
1 class InvalidAmount(Exception): 3 usages
2     def __init__(self, message):
3         super().__init__(message)
4
5 class Bank_Account: 1 usage
6     def __init__(self, balance=0):
7         self.__check_amount(balance)
8         self.__balance = balance
9
10    def deposit(self, amount): 1 usage
11        self.__check_amount(amount)
12        self.__balance += amount
13
14    def withdraw(self, amount): 1 usage
15        self.__check_amount(amount)
16        if self.__balance >= amount:
17            self.__balance -= amount
18        else:
19            print('Error: Insufficient funds')
20
21    def get_balance(self):
22        return self.__balance
23
24    def __check_amount(self, amount): 3 usages
25        try:
26            amount = float(amount)
27        except ValueError:
28            raise InvalidAmount('Amount must be a number')
29        if amount < 0:
30            raise InvalidAmount('Amount must be greater than zero')
31
32    def __str__(self):
33        return f'The balance is ${self.__balance:,.2f}'
```

```
1 import bank_account
2
3 def main(): 1 usage
4
5     try:
6         my_account = bank_account.Bank_Account("#abc1000")
7         print(my_account)
8
9         print("Depositing $500:")
10        my_account.deposit(500)
11        print(my_account)
12
13        print("Withdrawing $200:")
14        my_account.withdraw(200)
15        print(my_account)
16
17    except bank_account.InvalidAmount as e:
18        print(f"Invalid amount error: {e}")
19
20 if __name__ == '__main__':
21     main()
```

Invalid amount error: Amount must be a number

Process finished with exit code 0

Customized Exception

```
1 class InvalidAmount(Exception): 3 usages
2     def __init__(self, message, operation_name):
3         super().__init__(message)
4         self.__operation_name = operation_name
5
6     def __str__(self):
7         return f'InvalidAmount in {self.__operation_name}: {super().__str__()}'
8
9 class Bank_Account: 1 usage
10     def __init__(self, balance=0):
11         self.__check_amount(balance, func: "Opening account")
12         self.__balance = balance
13
14     def deposit(self, amount): 1 usage
15         self.__check_amount(amount, func: "deposit")
16         self.__balance += amount
17
18     def withdraw(self, amount): 1 usage
19         self.__check_amount(amount, func: "withdraw")
20         if self.__balance >= amount:
21             self.__balance -= amount
22         else:
23             print('Error: Insufficient funds')
24
25     def get_balance(self):
26         return self.__balance
27
28     def __check_amount(self, amount, func): 3 usages
29         try:
30             amount = float(amount)
31         except ValueError:
32             raise InvalidAmount(message: 'Amount must be a number', func)
33         if amount < 0:
34             raise InvalidAmount(message: 'Amount must be greater than zero', func)
35
36     def __str__(self):
37         return f'The balance is ${self.__balance:,.2f}'
```

Customized Exception

```
1 class InvalidAmount(Exception): 3 usages
2     def __init__(self, message, operation_name):
3         super().__init__(message)
4         self.__operation_name = operation_name
5
6     def __str__(self):
7         return f'InvalidAmount in {self.__operation_name}: {super().__str__()}'
8
9 class Bank_Account: 1 usage
10     def __init__(self, balance=0):
11         self.__check_amount(balance, func: "Opening account")
12         self.__balance = balance
13
14     def deposit(self, amount): 1 usage
15         self.__check_amount(amount, func: "deposit")
16         self.__balance += amount
17
18     def withdraw(self, amount): 1 usage
19         self.__check_amount(amount, func: "withdraw")
20         if self.__balance >= amount:
21             self.__balance -= amount
22         else:
23             print('Error: Insufficient funds')
24
25     def get_balance(self):
26         return self.__balance
27
28     def __check_amount(self, amount, func): 3 usages
29         try:
30             amount = float(amount)
31         except ValueError:
32             raise InvalidAmount(message: 'Amount must be a number', func)
33         if amount < 0:
34             raise InvalidAmount(message: 'Amount must be greater than zero', func)
35
36     def __str__(self):
37         return f'The balance is ${self.__balance:,.2f}'
```

```
1 import bank_account
2
3 def main(): 1 usage
4
5     try:
6         my_account = bank_account.Bank_Account(1000)
7         print(my_account)
8
9         print("Depositing $500:")
10        my_account.deposit(500)
11        print(my_account)
12
13        print("Withdrawing $-200:")
14        my_account.withdraw(-200)
15        print(my_account)
16
17    except bank_account.InvalidAmount as e:
18        print(e)
19
20 if __name__ == '__main__':
21     main()
```

```
The balance is $1,000.00
Depositing $500:
The balance is $1,500.00
Withdrawing $-200:
InvalidAmount in withdraw: Amount must be greater than zero

Process finished with exit code 0
```

Customized Exception

```
1 class InvalidAmount(Exception): 3 usages
2     def __init__(self, message, operation_name):
3         super().__init__(message)
4         self.__operation_name = operation_name
5
6     def __str__(self):
7         return f'InvalidAmount in {self.__operation_name}: {super().__str__()}'
8
```

```
9 class Bank_Account: 1 usage
10     def __init__(self, balance=0):
11         self.__check_amount(balance, func: "Opening account")
12         self.__balance = balance
13
14     def deposit(self, amount): 1 usage
15         self.__check_amount(amount, func: "deposit")
16         self.__balance += amount
17
18     def withdraw(self, amount): 1 usage
19         self.__check_amount(amount, func: "withdraw")
20         if self.__balance >= amount:
21             self.__balance -= amount
22         else:
23             print('Error: Insufficient funds')
24
25     def get_balance(self):
26         return self.__balance
27
28     def __check_amount(self, amount, func): 3 usages
29         try:
30             amount = float(amount)
31         except ValueError:
32             raise InvalidAmount(message: 'Amount must be a number', func)
33         if amount < 0:
34             raise InvalidAmount(message: 'Amount must be greater than zero', func)
35
36     def __str__(self):
37         return f'The balance is ${self.__balance:,.2f}'
```

```
1 import bank_account
2
3 def main(): 1 usage
4
5     try:
6         my_account = bank_account.Bank_Account("#!#abc1000")
7         print(my_account)
8
9         print("Depositing $500:")
10        my_account.deposit(500)
11        print(my_account)
12
13        print("Withdrawing $200:")
14        my_account.withdraw(200)
15        print(my_account)
16
17    except bank_account.InvalidAmount as e:
18        print(e)
19
20 if __name__ == '__main__':
21     main()
```

```
InvalidAmount in Opening account: Amount must be a number
Process finished with exit code 0
```

Introduction to GitHub Copilot

- **What is GitHub Copilot?**
 - Gen AI-powered code completion tool designed to assist developers.
 - Suggests code snippets and functions based on work context, enhancing coding speed.
- **Technology Behind GitHub Copilot:**
 - Based on OpenAI's Codex model.
 - Understands multiple programming languages, generates code snippets, tests, and algorithms.
- **Audience:**
 - Junior Engineers: Aids in learning, acting as a digital mentor.
 - Senior Engineers: Accelerates coding and simplifies complex tasks.

Introduction to GitHub Copilot

- **Huge Impact**
 - [Research](#) has demonstrated that it enhances developer productivity and boosts overall happiness.
- **Installation:**
 - Installable as an extension in supported IDEs: PyCarm, Visual Studio, Visual Studio Code, JetBrains & Vim.

When using GitHub Copilot...

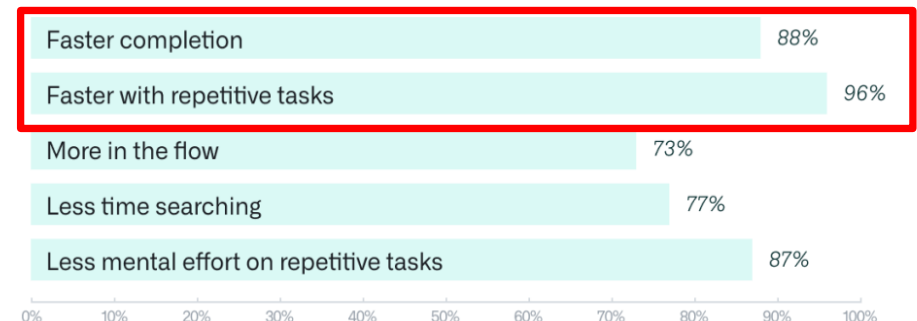
Perceived Productivity



Satisfaction and Well-being*



Efficiency and Flow*



Main Features of GitHub Copilot

- **Code generation**
 - Copilot generates code in real-time, based on the comments you write and the code you've already typed. It adapts to the coding style and preferences of the user.
- **Language and Framework Support:**
 - It supports a wide range of programming languages including C, C++, Java, Python, JavaScript, TypeScript, Ruby, and more, along with various frameworks.
- **Learning from Context:**
 - It learns from the context within your existing code - meaning the more descriptive your code and comments, the more accurate the suggestions.
- **Licensing and Resources:**
 - Requires a license; free for students and educators via [GitHub Education](#).
 - For more details: [GitHub Copilot Official Website](#)
 - A Beginner's Guide to Prompt Engineering with GitHub Copilot: [You can access the guide here.](#)



Sign in to GitHub Education for Students



לינק להסבר כיצד נרשמים לחשבון GitHub כסטודנטים

<https://docs.github.com/en/education/explore-the-benefits-of-teaching-and-learning-with-github-education/github-education-for-students/apply-to-github-education-as-a-student>

סרטון יוטיוב המסביר כיצד נרשמים לחשבון gitHub
כסטודנטים

<https://www.youtube.com/watch?v=iyThW3RvZpk>



YouTube

Start working with GitHub Copilot

Write a function from a
comment and then
press **Enter**

```
#The factorial function uses recursion to  
# calculate the factorial of its argument,  
# which is assumed to be nonnegative.  
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

To accept press **Tab**
To decline press **Esc**

```
#The factorial function uses recursion to  
# calculate the factorial of its argument,  
# which is assumed to be nonnegative.  
3 usages  
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)|
```

Start working with GitHub Copilot

Write a function from a
declaration

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

To accept press **Tab**
To decline press **Esc**

Start working with GitHub Copilot

Write a class from a
declaration

```
class BankAccount:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance
```

To accept press **Tab**
To decline press **Esc**

Press Enter to continue
getting suggestions

```
class BankAccount:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        return self.balance

    def withdraw(self, amount):
        if amount > self.balance:
            return "Insufficient funds"
        else:
            self.balance -= amount
            return self.balance

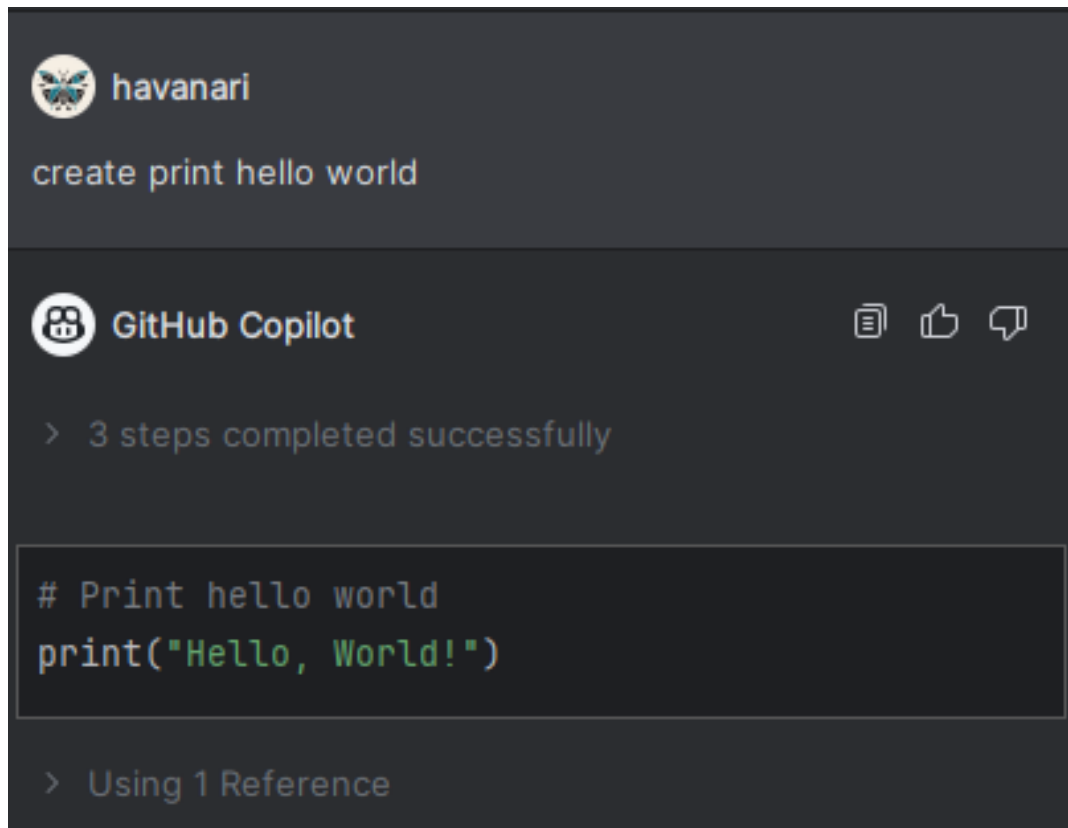
    def __str__(self):
        return f"Account Holder: {self.account_holder}\nBalance: {self.balance}"
```

Introduction to GitHub Copilot Chat

- **Overview:**
 - With Copilot Chat, you have the ability to engage in conversations with the tool.
 - It's like having a helpful conversation partner for your coding challenges
- **Use Cases:**
 - Ask questions related to your code
 - Seek explanations for code segments
 - Request assistance with debugging tasks
 - Exploring innovative approaches to problem-solving
 - Comprehending legacy code **and more...**
- **Installation:**
 - Installable as an extension in supported IDEs: PyCharm, Visual Studio Code and Visual Studio. JetBrains IDE is currently in beta,
- **Resources:**
 - For more details: [GitHub Copilot Chat Official Website](#)



Copilot Chat Can Generate Code



Copilot Chat Can Explain Code

Highlight the code you wish to understand

```
# Print hello world
print("Hello, World!")

#make a new line asking for the user's name
name = input("What is your name? ")

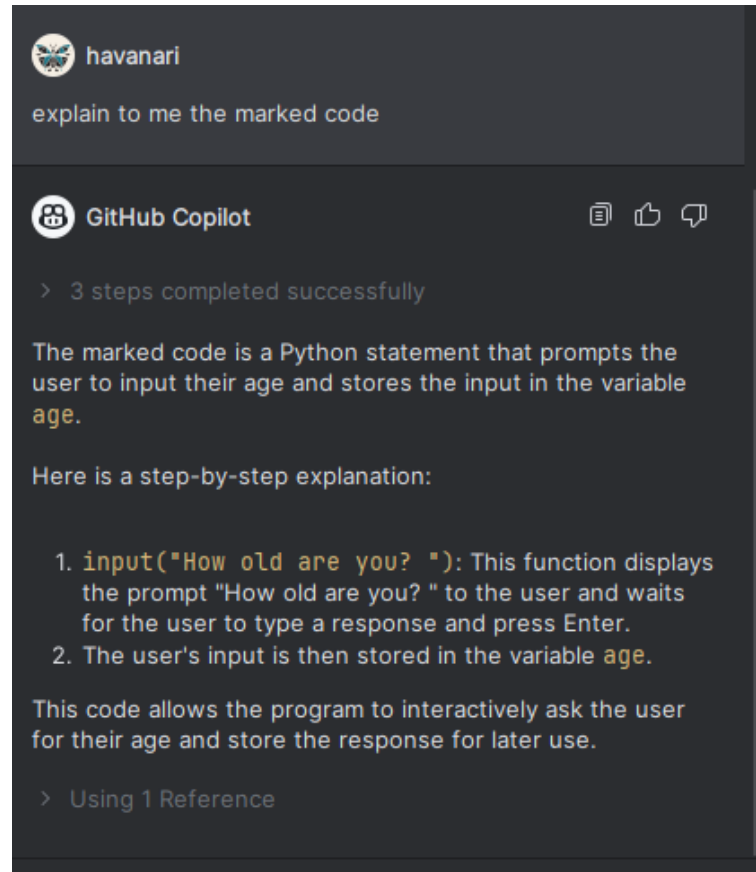
#Print the user's name
print("Hello, " + name + "!")

#ask the user for their age
age = input("How old are you? ")

#Print the user's age
print("You are " + age + " years old.")

#The factorial function uses recursion to
# calculate the factorial of its argument,
# which is assumed to be nonnegative.
3 usages
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)
3 usages
```

Copilot Chat explanation



The screenshot shows the GitHub Copilot Chat interface. At the top, a user named 'havanari' has sent the message 'explain to me the marked code'. Below this, the GitHub Copilot bot has responded. It indicates that '3 steps completed successfully'. The main response states: 'The marked code is a Python statement that prompts the user to input their age and stores the input in the variable **age**.' It then provides a 'step-by-step explanation' with two numbered points: 1. 'input("How old are you? ")': This function displays the prompt "How old are you? " to the user and waits for the user to type a response and press Enter. 2. 'The user's input is then stored in the variable **age**.' Below this, it concludes: 'This code allows the program to interactively ask the user for their age and store the response for later use.' At the bottom, there is a link 'Using 1 Reference'.

Copilot Chat Can find mistakes and fixed them

Change the base case in the recursive factorial function

```
#write a recursive function that calculates the factorial of a number
3 usages
def factorial(n):
    if n == 2:
        return 1
    else:
        return n * factorial(n - 1)
```

There is a bug in the code, what is it?



havanari

there is a bug in the code, where is it?



GitHub Copilot



> 3 steps completed successfully

The bug in the selected code is in the base case of the `factorial` function. The base case should return 1 when `n` is 0 or 1, not 2. Here is the corrected version:

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)
```

> Using 1 Reference

Copilot Chat for Learning

 havanari

Explain the concept of recursion in Python with examples

 GitHub Copilot   

> 3 steps completed successfully

Recursion in Python is a programming technique where a function calls itself in order to solve a problem. The function typically has a base case that stops the recursion and a recursive case that continues the recursion.

Example 1: Factorial Calculation

The factorial of a number n is the product of all positive integers less than or equal to n . It can be defined recursively as:

- `factorial(0) = 1` (base case)
- `factorial(n) = n * factorial(n - 1)` (recursive case)

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)  
  
# Example usage  
print(factorial(5)) # Output: 120
```

Example 2: Fibonacci Sequence

The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones. It can be defined recursively as:

- `fibonacci(0) = 0` (base case)
- `fibonacci(1) = 1` (base case)
- `fibonacci(n) = fibonacci(n - 1) + fibonacci(n - 2)` (recursive case)

```
def fibonacci(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fibonacci(n - 1) + fibonacci(n - 2)  
  
# Example usage  
print(fibonacci(6)) # Output: 8
```

Key Points

- **Base Case:** The condition under which the recursion stops.
- **Recursive Case:** The part of the function where the function calls itself with a modified argument.

Recursion is useful for problems that can be broken down into smaller, similar subproblems. However, it is important to ensure that the base case is reached to avoid infinite recursion.

> Using 1 Reference

Explain the
concept of
recursion in
Python with
examples

Main Features of GitHub Copilot Chat

- **Generating unit test cases:**
 - Copilot Chat helps you create unit test cases by generating code snippets based on the code in your editor or the code snippet you highlight.
- **Explaining code and suggesting improvements:**
 - It can describe selected code using natural language and suggest enhancements, including error handling, logical flow improvements, and code readability.
- **Proposing code fixes:**
 - Copilot Chat offers bug-fixing suggestions by proposing code snippets and solutions based on the context of the error or issue, aiding in problem-solving.
- **Answering coding questions & Learning:**
 - You can seek assistance and clarification on coding problems from Copilot Chat, receiving responses in either natural language or code snippet format.
- **Language Translation to Other Programming Languages.**
- **Edits multiple files and writes unit testing ([YouTube link](#)).**
- **Code Review & Refactoring**

Copilot Limitations

- **Does GitHub Copilot “copy/paste”?**
 - No, GitHub Copilot generates suggestions using probabilistic reasoning.
- **Limited Configurability**
 - Github Co-pilot doesn't have any model options. The only way we can influence the output is with:
 - More extensive comments.
 - Function/Class names.
 - The existing code within your project.
 - Adding types to your code.
 - Using different packages.
- **And more...**